Unit 4 Report: Secure 3-Tier Web Application on AWS

Name: Sumit Mhalaskar **PRN:** 2124UCSM1065 **Topic:** Responsibility Mapping, Asset Definition and IAM Least Privilege Enforcement

1. Introduction

This report covers two connected parts of a cloud security project. The first part designs the architecture of a 3-tier web application (Web, App and Database tiers) on AWS, defines its assets and maps security responsibilities between the customer and the cloud provider using a Shared Responsibility Matrix. The second part builds a strict Identity and Access Management (IAM) structure based on the Principle of Least Privilege, with MFA enforced for administrators and an IAM Role that lets an EC2 instance reach S3 without hardcoded credentials.

2. Part 1: Responsibility Mapping and Asset Definition

2.1 Application Architecture

The application follows a standard 3-tier design inside one VPC.

Tier	AWS Components	Placement	Purpose
Web tier	Application Load Balancer, EC2 web servers (Nginx) in an Auto Scaling group	Public subnets (2 Availability Zones)	Receives HTTPS traffic from users and forwards it to the app tier
Application tier	EC2 instances running the business logic (Node.js/Python)	Private subnets	Processes requests, talks to the database and to S3
Database tier	Amazon RDS (MySQL/PostgreSQL), Multi-AZ	Private DB subnets	Stores persistent application data

Supporting services: Amazon S3 (static assets, uploads and logs), AWS IAM, CloudWatch, CloudTrail, AWS KMS, NAT Gateway and an Internet Gateway.

Traffic flow: User → Internet Gateway → ALB (public subnet) → Web EC2 → App EC2 (private subnet) → RDS (private DB subnet). The app tier reaches S3 through an IAM Role, and outbound internet access from private subnets goes through the NAT Gateway.

Network security design:

- Security Group for ALB: allows inbound 443 from 0.0.0.0/0.
- Security Group for Web tier: allows inbound 80/443 only from the ALB security group.
- Security Group for App tier: allows inbound app port (for example 8080) only from the Web tier security group.
- Security Group for DB tier: allows inbound 3306/5432 only from the App tier security group.
- Network ACLs add a stateless subnet-level layer; the DB subnets have no route to the Internet Gateway.

2.2 Asset Definition

Asset	Type	Sensitivity	Why it matters
Customer and application data in RDS	Data	High	Confidentiality and integrity of user records
S3 buckets (uploads, backups, logs)	Data	Medium to High	May hold user files and audit logs
Application source code and configuration	Software	High	Contains business logic and possibly secrets
EC2 instances (web and app)	Compute	Medium	Entry points that could be compromised
IAM users, groups, roles, policies	Identity	Critical	Control who can do what in the account
Encryption keys (KMS)	Cryptographic	Critical	Protect data at rest
CloudTrail and CloudWatch logs	Audit	High	Needed for detection and forensics

Asset	Type	Sensitivity	Why it matters
VPC, subnets, security groups	Network	High	Define the network attack surface

2.3 Shared Responsibility Matrix

AWS is responsible for security **of** the cloud. The customer (me, as the application owner) is responsible for security **in** the cloud. For this application the split is as follows.

Layer / Control	AWS Responsibility	Customer Responsibility (this application)
Physical data centres, power, cooling, hardware	Fully AWS	None
Global network infrastructure, Availability Zones, Regions	Fully AWS	Choosing Regions and AZs for resilience
Hypervisor and host virtualization layer	Fully AWS	None
Managed service infrastructure (RDS engine hosting, S3 durability, ALB service)	AWS operates and patches the underlying service	Configuring the service securely (public access, versions, parameters)
Guest OS on EC2 (Web and App tiers)	None	Patching, hardening, removing unused services, anti-malware
Application code and dependencies	None	Secure coding, dependency scanning, input validation, secrets handling
Database engine patching on RDS	AWS applies patches in maintenance windows	Choosing maintenance windows, parameter groups, minor-version upgrades
Database users, passwords and access rules	None	Strong credentials, least-privilege DB users, no public access

Layer / Control	AWS Responsibility	Customer Responsibility (this application)
VPC design, subnets, route tables	Provides the VPC capability	Designing public/private separation, routing
Security Groups and NACLs	Provides the features	Writing and reviewing rules (tier-to-tier only)
Encryption at rest	Provides KMS and encryption features	Enabling encryption on EBS, RDS and S3, managing key policies
Encryption in transit	Provides TLS endpoints and ACM	Issuing certificates and enforcing HTTPS/TLS end to end
IAM service availability	Operates IAM	Designing users, groups, roles, policies, MFA, access key hygiene
Logging and monitoring	Provides CloudTrail, CloudWatch	Enabling, retaining, alerting and reviewing logs
Backup and recovery	Provides snapshot and replication features	Backup schedule, retention, restore testing
Compliance of the application	AWS holds infrastructure certifications	Meeting compliance for data and processes in the application
DDoS protection (basic)	AWS Shield Standard is automatic	Optionally adding AWS WAF rules and Shield Advanced

Key takeaway: AWS secures everything up to the hypervisor and managed service internals. Everything above that, namely operating systems, applications, data, network configuration and identities, remains my responsibility.

3. Part 2: IAM and Least Privilege Enforcement

3.1 Design Overview

Access is organised around three IAM Groups, each with a custom policy that grants only the actions its members need. Permissions are never attached directly to individual users.

Group	Members	Purpose	Key permissions
Admins	Cloud administrators	Manage infrastructure and identities	Broad management of project services, with MFA required
Developers	Application developers	Deploy and troubleshoot the app	Limited EC2, S3 (app bucket) and log access; no IAM changes
Auditors	Security and compliance reviewers	Review configuration and activity	Read-only access to configuration, IAM and logs

An important rule: no one uses the root account for daily work, and root also has MFA enabled.

3.2 Custom Policy: Admins

This policy gives administrators control over the services used by the application, and still excludes dangerous account-level actions such as closing the account or changing billing.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ManageProjectServices",
      "Effect": "Allow",
      "Action": [
        "ec2:*",
        "elasticloadbalancing:*",
        "autoscaling:*",
        "rds:*",
        "s3:*",
        "cloudwatch:*",
        "logs:*",
```

```
        "kms:*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ManageIdentitiesForProject",
      "Effect": "Allow",
      "Action": [
        "iam:Get*",
        "iam:List*",
        "iam:CreateUser",
        "iam:CreateRole",
        "iam:CreateGroup",
        "iam:CreatePolicy",
        "iam:AttachGroupPolicy",
        "iam:AttachRolePolicy",
        "iam:AddUserToGroup",
        "iam:PassRole"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyAccountLevelChanges",
      "Effect": "Deny",
      "Action": [
        "account:*",
        "aws-portal:*",
        "organizations:LeaveOrganization"
      ],
      "Resource": "*"
    }
  ]
}
```

3.3 Custom Policy: Developers

Developers can manage only EC2 instances tagged for the project, read and write only the application bucket, and read logs. IAM is not available to them.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DescribeEC2",
      "Effect": "Allow",
      "Action": ["ec2:Describe*"],
      "Resource": "*"
    },
    {
      "Sid": "ControlTaggedInstancesOnly",
      "Effect": "Allow",
      "Action": ["ec2:StartInstances", "ec2:StopInstances",
"ec2:RebootInstances"],
      "Resource": "arn:aws:ec2:*:*:instance/*",
      "Condition": {
        "StringEquals": {"aws:ResourceTag/Project": "ThreeTierApp"}
      }
    },
    {
      "Sid": "AppBucketReadWrite",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::threetier-app-bucket/*"
    },
    {
      "Sid": "AppBucketList",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::threetier-app-bucket"
    },
    {
      "Sid": "ReadLogs",
      "Effect": "Allow",
      "Action": ["logs:GetLogEvents", "logs:DescribeLogStreams",
"logs:FilterLogEvents", "cloudwatch:GetMetricData"],
      "Resource": "*"
    },
    {
      "Sid": "DenyIAMAndDeletes",
      "Effect": "Deny",
      "Action": ["iam:*", "ec2:TerminateInstances", "rds:DeleteDBInstance",
"s3:DeleteBucket"],
```

```
      "Resource": "*"
    }
  ]
}
```

3.4 Custom Policy: Auditors

Auditors get read-only visibility into configuration, identities and activity history, and cannot change anything.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOnlyConfiguration",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "rds:Describe*",
        "elasticloadbalancing:Describe*",
        "s3:GetBucketPolicy",
        "s3:GetBucketAcl",
        "s3:GetEncryptionConfiguration",
        "s3:GetBucketPublicAccessBlock",
        "s3:ListAllMyBuckets",
        "kms:Describe*",
        "kms:List*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadIdentityAndAuditData",
      "Effect": "Allow",
      "Action": [
        "iam:Get*",
        "iam:List*",
        "iam:GenerateCredentialReport",
        "cloudtrail:LookupEvents",
        "cloudtrail:DescribeTrails",
        "cloudtrail:GetTrailStatus",
        "logs:Describe*",
        "logs:GetLogEvents"
```

```
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyAllWrites",
      "Effect": "Deny",
      "Action": [
        "ec2:Run*", "ec2:Terminate*", "ec2:Modify*",
        "rds:Create*", "rds:Delete*", "rds:Modify*",
        "s3:Put*", "s3:Delete*",
        "iam:Create*", "iam:Delete*", "iam:Attach*", "iam:Put*"
      ],
      "Resource": "*"
    }
  ]
}
```

3.5 MFA Enforcement for the Admin Group

MFA is enforced with an additional custom policy attached to the Admins group. It denies every action, except the ones needed to set up an MFA device, when the user has not signed in with MFA. An admin without MFA can therefore only manage their own MFA device and nothing else.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSelfMFAManagement",
      "Effect": "Allow",
      "Action": [
        "iam:ListVirtualMFADevices",
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ResyncMFADevice",
        "iam:ListMFADevices",
        "iam:ChangePassword",
        "iam:GetUser"
      ],
      "Resource": [
        "arn:aws:iam::*:mfa/${aws:username}",
        "arn:aws:iam::*:user/${aws:username}"
```

```
      ]
    },
    {
      "Sid": "DenyEverythingElseWithoutMFA",
      "Effect": "Deny",
      "NotAction": [
        "iam:ListVirtualMFADevices",
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ResyncMFADevice",
        "iam:ListMFADevices",
        "iam:ChangePassword",
        "iam:GetUser",
        "sts:GetSessionToken"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {"aws:MultiFactorAuthPresent": "false"}
      }
    }
  ]
}
```

Because an explicit Deny always overrides an Allow, the Admin custom policy only takes effect after the user authenticates with MFA.

3.6 IAM Role for EC2 to Access S3 (No Hardcoded Credentials)

Instead of storing access keys in code or on the server, the app-tier EC2 instances assume an IAM Role through an instance profile. AWS delivers short-lived temporary credentials automatically through the instance metadata service, and the AWS SDK picks them up without any configuration.

Role name: `EC2-AppTier-S3-Access-Role`

Trust policy (who may assume the role: only the EC2 service):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"Service": "ec2.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Permissions policy (what the role may do: only the application bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ObjectAccess",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::threetier-app-bucket/*"
    },
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::threetier-app-bucket"
    }
  ]
}
```

Implementation steps:

1. In IAM, create the role with trusted entity type AWS service and use case EC2.
2. Attach the custom permissions policy above.
3. Name the role `EC2-AppTier-S3-Access-Role` (AWS creates a matching instance profile).
4. In the EC2 console, select the app-tier instance and choose Actions, Security, Modify IAM role, then attach the role. For the Auto Scaling group, add it to the launch template.
5. Do not run `aws configure` and do not store access keys anywhere on the instance.

3.7 Group Setup Steps

1. Create the three groups: `Admins`, `Developers`, `Auditors`.
2. Create the custom policies from sections 3.2 to 3.5 (for example `Admin-ProjectAccess`, `Admin-Require-MFA`, `Dev-LimitedAccess`, `Auditor-ReadOnly`).
3. Attach `Admin-ProjectAccess` and `Admin-Require-MFA` to Admins, `Dev-LimitedAccess` to Developers, and `Auditor-ReadOnly` to Auditors.
4. Create individual IAM users and add each to exactly one group.
5. Require every user to register an MFA device, with the policy above making it mandatory for Admins.
6. Set a strong account password policy (minimum length, complexity, rotation).

4. Testing and Verification

Test	Expected Result
Admin signs in without MFA and tries to launch an EC2 instance	Denied by the MFA policy
Admin signs in with MFA and launches an EC2 instance	Allowed
Developer tries to create an IAM user	Denied (explicit deny)
Developer uploads a file to `threetier-app-bucket`	Allowed
Developer reads a different S3 bucket	Denied (no matching allow)
Auditor lists IAM users and views CloudTrail events	Allowed
Auditor tries to stop an EC2 instance	Denied
App-tier EC2 runs `aws s3 ls s3://threetier-app-bucket` with no stored keys	Allowed through the role
App-tier EC2 tries to access a different bucket or delete the bucket	Denied
Search EC2 filesystem and code repository for access keys	None found

The IAM Policy Simulator and CloudTrail event history can be used to confirm each result.

5. How the Two Parts Connect

The Shared Responsibility Matrix shows that identity and access management, network configuration and data protection are customer duties. Part 2 fulfils the identity portion of that responsibility: groups and custom policies enforce least privilege for humans, MFA protects the most powerful accounts, and the EC2 role removes long-lived credentials from the application tier. Together they reduce the attack surface and limit the damage if any single credential or server is compromised.

6. Conclusion

The 3-tier architecture separates the Web, App and Database layers across public and private subnets, and the Shared Responsibility Matrix makes clear that AWS secures the underlying cloud while I secure the operating systems, application, data, network rules and identities. The IAM design applies the Principle of Least Privilege through three purpose-built groups with custom policies, mandatory MFA for administrators, and a custom EC2 role that grants secure S3 access with temporary credentials instead of hardcoded keys. This combination gives a practical, auditable security baseline for the application.

7. References

- AWS Shared Responsibility Model, AWS documentation
- AWS IAM User Guide: policies, groups, roles and MFA
- AWS Well-Architected Framework, Security Pillar
- AWS documentation: IAM roles for Amazon EC2